

Grounding Stratum in Established Research: A Terminology Mapping

The eight specification validity properties defined in the Stratum framework have deep roots in academic literature, industry standards, and formal methods. Each property maps to established concepts with **40+ years of foundational research**, enabling rigorous grounding in proven theory while providing precise terminology for cross-disciplinary communication.

The core terminology mapping

Every Stratum property corresponds to established terms across formal methods, requirements engineering, enterprise architecture, and safety-critical standards. The table below synthesizes findings from IEEE/ISO standards, SEI publications, OMG specifications, and foundational computer science literature.

Stratum Property	Primary Established Term	Secondary Terms	Key Standard/Paper
WellFormed(S)	Well-formedness	Type correctness, Syntactic validity, Metamodel conformance	OMG OCL 2.4, ISO Z Standard, Wright-Felleisen 1994
Provenanced(S)	Bidirectional traceability	Requirements provenance, Refinement relation, Rationale	IEEE 29148:2018, DO-178C, Gotel & Finkelstein 1994
Fulfilled(S)	Constraint satisfaction	Model satisfaction ($M \models S$), Internal completeness, Adequacy	IEEE 29148:2018, Benveniste et al. 2018
NonContradictory(S)	Internal consistency	Satisfiability, Non-contradiction, Invariant preservation	VDM Proof Obligations, Lamport TLA+ 2002
Materialized(S)	Reification/Refinement	Realization, Concretization, Implementation binding	Jones VDM 1990, ArchiMate 3.2, DO-178C
Predictable(S)	Modifiability	Change impact analysis, Tractable IA, Ripple effect	SEI CMU/SEI-2007-TR-002, Bohner & Arnold 1996
Stratified(S)	Layered abstraction	Dependency rule, Separation of concerns, CIM/PIM/PSM	Martin Clean Architecture 2017, OMG MDA, MOF M0-M3
Diagnosable(S)	Minimal unsatisfiable subset	Unsat core, Minimal conflict set, Diagnosability	Reiter 1987, Junker QuickXplain 2004, Sampath 1995

WellFormed(S) traces to syntactic validity theory

The concept of well-formedness appears in virtually every formal specification language, serving as the decidable precondition for semantic analysis. **Z notation** defines well-formedness through signature validity where the theory of signatures is decidable—common variables must have identical types. [University of Oxford](#) VDM-SL standardization (ISO) provides "well-formedness predicates" for increasingly rigorous type analyses, distinguishing between obviously ill-typed and obviously well-typed expressions. [ResearchGate](#)

The OMG OCL 2.4 Specification explicitly states: "OCL is a typed language so that each OCL expression has a type. To be **well formed**, an OCL expression must conform to the **type conformance rules** of the language."

[omg](#) [OMG](#) Multiple sections (8.3.7, 10.2.2, 11.3-11.7) define well-formedness rules. The MOF 2.5.1 Specification uses constraints to define "well-formedness rules for the metadata described by a metamodel." [University of Ottawa](#) [OMG](#)

In programming language theory, the Wright-Felleisen formulation provides the canonical definition via Progress and Preservation theorems: well-typed programs do not "go wrong" during execution. [SIGPLAN Blog](#) IEEE 29148:2018 extends this to requirements with nine characteristics for "well-formed textual requirements" including necessary, unambiguous, complete, singular, and verifiable. [ResearchGate](#)

Key distinction: All WellFormed concepts are designed to be mechanically checkable (decidable), distinguishing them from semantic validity which may be undecidable.

Provenanced(S) aligns with traceability research

The IEEE Systems and Software Engineering Vocabulary provides three definitions of traceability, with the third most directly analogous to Provenanced(S): "**The degree to which each element in a software development product establishes its reason for existing.**" [Wikipedia](#) This precisely captures the Stratum requirement that every element traces back to a root justification.

ISO/IEC/IEEE 29148:2018 defines a traceable requirement as one that is "upwards traceable to specific documented stakeholder statement(s) of need... and downwards traceable to the specific requirements in the lower tier requirements specification." The key property is **bidirectionality**: forward traceability ensures all requirements are implemented; backward traceability ensures nothing exists without justification. [Grokipedia](#)

Safety-critical standards enforce this rigorously. DO-178C requires that "**every line of code must be traceable to a requirement**" with Table A-7, Objective 9 explicitly addressing "verification of additional code that cannot be traced to Source Code." IEC 61508-3 Annex A.1 specifies that backward traceability checks "that the implementation is **justified** by a requirement"—the core provenance property.

In formal methods, the refinement calculus (Back & von Wright 1998) provides mathematical foundations where each refinement step is proven to preserve specification truth. Gotel & Finkelstein's 1994 paper "An Analysis of the Requirements Traceability Problem" remains the foundational academic reference.

Fulfilled(S) corresponds to constraint satisfaction and completeness

The concept of a specification satisfying its own declared constraints maps to **model satisfaction** in formal logic ($M \models S$), **constraint satisfaction** in OCL/MDE, and **internal completeness** in requirements engineering. The distinction between internal and external completeness is critical: internal completeness means the specification is self-consistent and satisfies its declared invariants, while external completeness addresses adequacy relative to stakeholder needs.

IEEE 29148:2018 defines "complete" for individual requirements as "needs no further amplification because it is measurable and sufficiently describes the capability." For requirement sets, completeness means "no TBD/TBS/TBR clauses." DO-178C specifies 71 verification objectives organized by Design Assurance Level, with completeness requiring evidence that "each phase is complete, coherent, and traceable to its predecessor."

Contract-based design (Benveniste et al. 2018) formalizes this through assume-guarantee contracts where a component satisfies a contract if "all runs that satisfy assumptions A must satisfy guarantees G." The compositional theorem states that if all components satisfy their own contracts, a global system contract is satisfied—directly analogous to Fulfilled(S) for composite specifications.

The OMG OCL 2.4 Specification requires that "an OCL expression is an invariant of the type and must be true for all instances of that type at any time." (omg) (Modeling Languages) Tools like USE (UML-based Specification Environment) validate models by checking if instances satisfy all declared invariants.

NonContradictory(S) maps to consistency and satisfiability

Internal consistency—the absence of semantic conflicts—has rigorous formalizations across every specification domain. The classical **Law of Non-Contradiction** (Aristotle) states $\neg(P \wedge \neg P)$: a proposition cannot be both true and false simultaneously. In SAT/SMT solving, this becomes **satisfiability**: a set of constraints is consistent if and only if their conjunction is satisfiable.

VDM explicitly defines **proof obligations for internal consistency**: "A proof obligation is a theorem stating that a certain property must hold in order for a formal specification to be internally consistent. If a proof obligation can be proved, then the referred part in the specification is consistent." (Springer) Tools like VDMTools and Overture automate proof obligation generation.

For TLA+ specifications, Lamport defines **invariants** as "state predicates true of all reachable states," with TLC model checker performing breadth-first search to verify invariance. In Alloy, consistency analysis uses SAT-based model finding: a specification is consistent if at least one valid instance exists that satisfies all facts within a bounded scope.

IEEE 29148:2018 requires that requirements be "**consistent**: free of conflicts with other requirements." The KAOS methodology formally defines conflicts as goals that are "mutually inconsistent under domain axioms" and divergences as weaker conflicts occurring under boundary conditions. (Academia.edu)

SMT solvers (Z3, CVC5) formalize **T-satisfiability** where formulas are satisfiable if a model exists in theory T. When a T-solver finds inconsistency, it returns a **conflict set**—the minimal subset causing the contradiction—

directly supporting diagnosability.

Materialized(S) builds on refinement and realization theory

The grounding of abstract specifications in concrete artifacts has a 40-year history in **refinement calculus**. C.B. Jones's VDM work defines **reification** as "finding a more concrete representation of abstract data types"—stepwise transformation with proof obligations via retrieve functions. Woodcock & Davies formulate Z refinement theory showing how abstract specifications systematically refine to executable implementations.

The He-Hoare-Sanders formulation (1986) introduces **downward and upward simulation** with abstraction relations $\rho \subseteq \Sigma_A \times \Sigma_C$ between abstract and concrete states. **Joint completeness** using both simulation notions ensures sound refinement. In abstract interpretation (Cousot & Cousot), the **concretization function** γ maps abstract analysis results to actual program behavior.

Enterprise architecture uses the **realization relationship** (ArchiMate 3.2): "A dependency relationship that connects a logical entity with a more concrete entity that implements it." [The Open Group](#) The progression flows from Architecture Building Blocks (ABBs) to Solution Building Blocks (SBBs), with physical components being "tangible software components, devices" and artifacts describing "storage of information (files, database tables)." [The Open Group](#)

In MDA, the transformation chain $CIM \rightarrow PIM \rightarrow PSM \rightarrow \text{Code}$ represents progressive materialization.

[ScienceDirect](#) A specification is "fully grounded" when the Platform Specific Model produces deployable Executable Object Code. DO-178C requires bidirectional traceability to Executable Object Code with "100% requirements coverage" and "no orphan code."

Predictable(S) connects to modifiability and impact analysis

The SEI defines **modifiability** as "a quality attribute that relates to the cost of change and refers to the ease with which a software system can accommodate changes." The formal parameters include coupling strength (probability of modification propagation) and cohesion. The cost model: "Average cost of modifying a responsibility = direct modification cost + average cost of modifying all responsibilities with propagated modifications."

Bohner & Arnold's foundational 1996 work defines **change impact analysis** as "identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change." Two primary classes exist: traceability-based IA (relationships analyzed to determine scope) and dependency-based IA (linkages assessed for consequences).

For complexity guarantees, Reps, Horwitz, and Sagiv's POPL 1995 paper on **IFDS problems** provides "a polynomial-time algorithm for finding precise solutions to a general class of interprocedural dataflow-analysis problems." [UW Computer Sciences](#) When dataflow functions distribute over the meet operator and the domain is finite, the Tabulation Algorithm runs in $O(N \cdot D^2)$.

The incremental model transformation literature (VIATRA framework) comes closest to linear-time requirements, establishing that "synchronization time is proportional to the **magnitude of change** and not to the

size of models"—precisely the property Predictable(S) requires.

Yau & Collofello's **ripple effect analysis** (IEEE TSE 1985) measures structural stability upon modification using applied probabilities and matrix algebra. Low treewidth dependency graphs enable tractable reachability analysis.

Stratified(S) embodies the dependency rule and abstraction hierarchy

Robert C. Martin's Clean Architecture articulates **The Dependency Rule**: "Source code dependencies can only point inwards. Nothing in an inner circle can know anything at all about something in an outer circle."

(Clean Coder) (cleancoder) The layers (inner to outer) are Entities, Use Cases, Interface Adapters, and Frameworks & Drivers. "As you move inwards, the level of abstraction increases." (Clean Coder)

The **Dependency Inversion Principle** states: "High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details." (Stackify)

A crucial distinction from Abelson & Sussman's work on **stratified design** (1987): "Where layers are levels of equal abstraction in software, **strata are levels of different abstraction**." (Medium) Layers provide horizontal separation by functional concern; strata provide vertical separation by abstraction level.

ArchiMate's three core layers (Business, Application, Technology) formalize this with "higher layers making use of services provided by lower layers" (Visual-paradigm) and **realization relationships** connecting elements across layers. (The Open Group) TOGAF's Content Metamodel distinguishes conceptual, logical, and physical elements. (The Open Group)

OMG's **MOF four-layer metamodel architecture** (M0-M3) provides rigorous formalization: "Models at every level conform to a model belonging to the upper level." (Wikipedia) MOF is closed—M3 conforms to itself. MDA's CIM/PIM/PSM separation achieves vertical separation of concerns with transformations flowing downward. (Wikipedia)

The Zachman Framework explicitly notes: "The Rows define **TRANSFORMATION**, NOT decomposition"—reification through perspectives from contextual through instantiation.

Diagnosable(S) leverages MUS extraction and model-based diagnosis

The ability to produce actionable failure explanations with minimal conflict sets has rigorous foundations in SAT/SMT theory and model-based diagnosis. A **Minimal Unsatisfiable Subset (MUS)** is a subset M of an unsatisfiable formula F where M is unsatisfiable but removing any clause makes it satisfiable. (Springer) **Unsat cores** are the practical approximation returned by solvers.

Reiter's seminal 1987 paper "A Theory of Diagnosis from First Principles" establishes that "**the set of diagnoses is exactly the set of minimal hitting sets of the collection of all conflicts**." A conflict set is a set of components whose assumed correct functioning is inconsistent with observations. This **hitting set duality**—every MUS hits every MCS—enables computing repairs from conflicts.

Junker's **QuickXplain algorithm** (2004) efficiently computes preferred minimal conflicts using divide-and-conquer with $O(n \cdot \log(k+1) + k^2)$ consistency checks. This non-intrusive algorithm works with any propagation mechanism.

Diagnosability from discrete-event systems theory (Sampath et al. 1995) defines when faults can be unambiguously detected: fault f is diagnosable iff no two infinite runs exist with identical observable projections where f occurs in one but not the other. Cimatti et al. reduced diagnosability verification to symbolic model checking.

Delta debugging (Zeller 1999) provides **1-minimality** for failure-inducing inputs: a minimal configuration where removing any single element results in a passing test.

Foundational bibliography by property

WellFormed(S)

- **OMG OCL 2.4 Specification** (formal/2014-02-03) — Sections 8.3.7, 10.2.2, 11.3-11.7 define well-formedness rules
- **OMG MOF 2.5.1 Specification** (ISO/IEC 19508:2014) — Constraints as well-formedness rules
- **Wright, A. & Felleisen, M.** (1994). "A Syntactic Approach to Type Soundness." *Information and Computation*
- **Pierce, B.** (2002). *Types and Programming Languages*. MIT Press
- **ISO/IEC/IEEE 29148:2018** — Nine characteristics for well-formed requirements
- **Spivey, J.M.** (1992). *The Z Notation: Reference Manual*, 2nd ed. Prentice Hall

Provenanced(S)

- **Gotel, O. & Finkelstein, C.** (1994). "An Analysis of the Requirements Traceability Problem." *IEEE RE Conference*
- **IEEE 24765** — Systems and Software Engineering Vocabulary (traceability definition #3)
- **ISO/IEC/IEEE 29148:2018** — Bidirectional traceability requirements
- **RTCA DO-178C** (2011) — Traceability objectives, Table A-7
- **IEC 61508-3 Annex A.1** — Forward/backward traceability requirements
- **Back, R.J.R. & von Wright, J.** (1998). *Refinement Calculus: A Systematic Introduction*. Springer
- **Ramesh, B. & Jarke, M.** (2001). "Toward Reference Models for Requirements Traceability." *IEEE TSE*

Fulfilled(S)

- **ISO/IEC/IEEE 29148:2018** — Completeness criteria for requirements
- **RTCA DO-178C** — 71 verification objectives by DAL
- **Benveniste, A. et al.** (2018). "Contracts for System Design." *Foundations and Trends in EDA*
- **OMG OCL 2.4 Specification** — Invariant satisfaction semantics
- **Open Logic Project** — Model satisfaction relation ($M \models S$)

NonContradictory(S)

- **Jones, C.B.** (1990). *Systematic Software Development using VDM*. Prentice Hall — Proof obligations
- **Lamport, L.** (2002). *Specifying Systems: The TLA+ Language and Tools*. Addison-Wesley
- **Jackson, D.** (2012). *Software Abstractions: Logic, Language, and Analysis*. MIT Press — Alloy consistency
- **Barrett, C. et al.** "Satisfiability Modulo Theories." *Handbook of Satisfiability*
- **Van Lamsweerde, A.** "Managing Conflicts in Goal-driven Requirements Engineering"
- **Gogolla, M. et al.** — USE tool for OCL invariant validation

Materialized(S)

- **Jones, C.B.** (1990). *Systematic Software Development using VDM*. Prentice Hall — Reification
- **He, J., Hoare, C.A.R., Sanders, J.W.** (1986). "Data Refinement Refined." *ESOP*
- **Woodcock, J. & Davies, J.** (1996). *Using Z: Specification, Refinement, and Proof*. Prentice Hall
- **ArchiMate 3.2 Specification** — Realization relationship definition
- **OMG MDA Guide v2.0** — CIM/PIM/PSM transformation chain
- **Cousot, P. & Cousot, R.** — Abstract interpretation (α - γ functions)
- **Kruchten, P.** (1995). "Architectural Blueprints—The '4+1' View Model." *IEEE Software*

Predictable(S)

- **Bachmann, F., Bass, L., Nord, R.** (2007). "Modifiability Tactics." CMU/SEI-2007-TR-002
- **Bohner, S.A. & Arnold, R.S.** (1996). "Software Change Impact Analysis." IEEE
- **Reps, T., Horwitz, S., Sagiv, M.** (1995). "Precise Interprocedural Dataflow Analysis via Graph Reachability." *POPL*

- **Yau, S.S. & Collofello, J.S.** (1985). "Design Stability Measures for Software Maintenance." *IEEE TSE*
- **VIATRA Framework** — Incremental model transformation (change-proportional complexity)
- **TOGAF 9.2 Standard** — Phase H Architecture Change Management

Stratified(S)

- **Martin, R.C.** (2017). *Clean Architecture: A Craftsman's Guide*. Prentice Hall — The Dependency Rule
- **Abelson, H. & Sussman, G.** (1987). "Lisp: A Language for Stratified Design." *AI Memo*
- **ArchiMate 3.1 Specification** — Business/Application/Technology layers
- **TOGAF 9.2 Standard** — Architecture domains, ABB/SBB distinction
- **OMG MOF 2.5.1** — M0-M3 metamodel hierarchy
- **OMG MDA Guide** — CIM/PIM/PSM vertical separation
- **Zachman, J.** (1987). "A Framework for Information Systems Architecture." *IBM Systems Journal*

Diagnosable(S)

- **Reiter, R.** (1987). "A Theory of Diagnosis from First Principles." *Artificial Intelligence* 32(1):57-95
[Springer](#)
- **Junker, U.** (2004). "QUICKXPLAIN: Preferred Explanations and Relaxations." *AAAI-04*
[ACM Digital Library](#)
- **Liffiton, M.H. & Sakallah, K.A.** (2008). "Algorithms for Computing Minimal Unsatisfiable Subsets." *JSAT*
- **Sampath, M. et al.** (1995). "Diagnosability of Discrete-event Systems." *IEEE TAC* 40(9):1555-1575
- **Zeller, A. & Hildebrandt, R.** (2002). "Simplifying and Isolating Failure-Inducing Input." *IEEE TSE*
- **de Kleer, J. & Williams, B.C.** (1987). "Diagnosing Multiple Faults." *Artificial Intelligence* 32(1):97-130
[Academia.edu](#)
- **Marques-Silva, J. et al.** (2013). "On Computing Minimal Correction Subsets." *IJCAI*

Cross-domain patterns in specification validity

Three patterns emerge across all eight properties. First, **decidability stratification**: properties like WellFormed and NonContradictory have decidable checking procedures (type systems, SAT solvers), while others like Fulfilled and Provenanced require semi-decidable or process-based verification. Second, **the IEEE/OMG/safety-critical convergence**: IEEE 29148, OMG specifications (MOF, OCL, MDA), and safety

standards (DO-178C, ISO 26262, IEC 61508) independently arrived at nearly identical property definitions, suggesting these validity properties are fundamental. Third, **formal methods foundations**: every property maps to concepts with 30-40 years of formal methods research, from refinement calculus to model-based diagnosis, providing mathematical rigor for practical engineering.

The Stratum framework's eight properties thus represent a coherent synthesis of established specification theory, offering a unified vocabulary grounded in IEEE standards, OMG specifications, SEI publications, and foundational computer science research.